

SQL-I - Basics

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



MySQL Clients

- **Terminal / Command-line Client**
  - Program: mysql
  - For executing SQL directly
  - Fast, lightweight
  - Requires SQL knowledge
- **GUI Clients**
  - **Examples:**
    - **MySQL Workbench**
    - **DataGrip**
    - **TablePlus**
    - **VS Code SQL extensions**
  - **They provide:**
    - Editors + Run buttons
    - Data viewing
    - Schema visualization
    - Easy management tools

SQL(Structured Query Language)

- Declarative Language
  - You specify **what you want, not how to get it**
  - **Example**: "Give me all students with GPA > 3.5”
  - You don't specify how to search
- Based on **relational algebra**
- S.Q.L. or Sequel supported by all major commercial **relational database systems**
- Standardized – many new features over time

SQL - Basics

- Overview of The SQL Query Language
- Basic DDL Statements
- Basic DML Statements
  - Select-From-Where
  - Aggregation
  - Table Variables
  - Set Operators
  - Group-By

SQL: Introduction

- SQL-DDL (Data Definition Language)
  - create database x,
  - use x,
  - drop database x
  - …
- SQL-DML (Data Manipulation Language)
  - select,
  - insert,
  - delete,
  - update,
  - …
- Other Commands
  - indexes, constraints, views, triggers, transactions, authorization, …

SQL: Create, Use, Drop Databases

- Database Creation and Deletion

- SHOW DATABASES;
- drop database Admission_STD_ID;
- use Admission_STD_ID;

- SHOW TABLES;
- DESCRIBE student;
- SHOW COLUMNS FROM student;

SQL: Create Tables

- An SQL relation is defined using the **create table** command:

  create table *r* (A_1 D_1, A_2 D_2, ..., A_n D_n,
  (constraint$_1$),
  ...,
  (constraint$_k$))

  - *r* is the name of the relation
  - each A_i is an attribute name in the schema of relation *r*
  - D_i is the data type of values in the domain of attribute A_i

- Example:

```
create table instructor (
        ID          char(5),
        name        varchar(20),
        dept_name   varchar(20),
        salary      numeric(8,2),
                    Primary key (ID));
```

Basic Types in SQL

- **char(n).** Fixed length character string, with user-specified length *n.*
- **varchar(n).** Variable length character strings, with user-specified maximum length *n.*
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d).** Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric**(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real, double precision.** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.

Part of SQL-Server 2012 Data Types
https://www.connectionstrings.com/sql-server-2012-data-types-reference/

DATATYPE	MIN	MAX	STORAGE	TYPE	NOTES
Bigint	-2^63	2^63-1	8 bytes	Exact	
Int	-2,147,483,648	2,147,483,647	4 bytes	Exact	
Smallint	-32,768	32,767	2 bytes	Exact	
Tinyint	0	255	1 bytes	Exact	
Bit	0	1	1 to 8 bit columns in the same table requires a total of 1 byte, 9 to 16 bits = 2 bytes, etc...	Exact	
Decimal	-10^38+1	10^38–1	Precision 1-9 = 5 bytes, precision 10-19 = 9 bytes, precision 20-28 = 13 bytes, precision 29-38 = 17 bytes	Exact	The Decimal and the Numeric data type is exactly the same. Precision is the total number of digits. Scale is the number of decimals. For both the minimum is 1 and the maximum is 38.
Numeric	same as Decimal	same as Decimal	same as Decimal	Exact	
Money	-2^63 / 10000	2^63-1 / 10000	8 bytes	Exact	
Smallmoney	-214,748.3648	214,748.3647	4 bytes	Exact	
Float	-1.79E + 308	1.79E + 308	4 bytes when precision is less than 25 and 8 bytes when precision is 25 through 53	Approx	Precision is specified from 1 to 53.
Real	-3.40E + 38	3.40E + 38	4 bytes	Approx	Precision is fixed to 7.
Datetime	1753-01-01 00:00:00.000	9999-12-31 23:59:59.997	8 bytes	Datetime	If you are running SQL Server 2008 or later and need milliseconds precision, use datetime2(3) instead to save 1 byte.
Smalldatetime	1900-01-01 00:00	2079-06-06 23:59		Datetime	
Date	0001-01-01	9999-12-31	3 bytes	Datetime	
Time	00:00:00.0000000	23:59:59.9999999		Datetime	Specifying the precision is possible. TIME(3) will have milliseconds precision. TIME(7) is the highest and the default precision. Casting values

MySQL Data Types

DATATYPE	MIN	MAX	STORAGE	TYPE	NOTES
TINYINT	-128	127 (SIGNED) 0 – 255 (UNSIGNED)	1 byte	Exact	Small integers
SMALLINT	-32,768	32,767 (SIGNED) 0 – 65,535 (UNSIGNED)	2 bytes	Exact	Medium small integers
MEDIUMINT	-8,388,608	8,388,607 (SIGNED) 0 – 16,777,215 (UNSIGNED)	3 bytes	Exact	Less common integer size
INT / INTEGER	-2,147,483,648	2,147,483,647 (SIGNED) 0 – 4,294,967,295 (UNSIGNED)	4 bytes	Exact	Standard integer
BIGINT	-2^63	2^63-1 (SIGNED) 0 – 2^64-1 (UNSIGNED)	8 bytes	Exact	Large integers
DECIMAL(M,D)	-10^38+1	10^38-1	Varies: M ≤ 65, D ≤ 30 Storage: ~ (M+2) bytes	Exact	Fixed-point; precision M, scale D
NUMERIC(M,D)	same as DECIMAL	same as DECIMAL	same as DECIMAL	Exact	Synonym for DECIMAL
FLOAT(p)	~ -3.402823466E+38	~ 3.402823466E+38	4 bytes (p ≤ 24)	Approx	Single-precision float
DOUBLE / REAL	~ -1.7976931348623157E+308	~ 1.7976931348623157E+308	8 bytes	Approx	Double-precision float
DATE	'1000-01-01'	'9999-12-31'	3 bytes	Datetime	Only date (YYYY-MM-DD)
DATETIME(fsp)	'1000-01-01 00:00:00'	'9999-12-31 23:59:59'	8 bytes (no fractional sec)	Datetime	Optional fractional seconds (0–6)
TIMESTAMP(fsp)	'1970-01-01 00:00:01 UTC'	'2038-01-19 03:14:07 UTC'	4 bytes (no fractional sec)	Datetime	Stored in UTC, auto-update possible
TIME(fsp)	'-838:59:59'	'838:59:59'	3 bytes	Datetime	Time interval or day time
YEAR	1901	2155	1 byte	Datetime	YEAR(4) only
CHAR(n)	Fixed-length, 0–255 chars	Depends on charset	n bytes	String	Padded with spaces
VARCHAR(n)	Variable-length, 0–65535 chars	Depends on row size	L+1 or L+2 bytes	String	Length stored separately
TEXT / BLOB	0 – 65535 bytes (TEXT)	Varies by type	Varies	String/Binary	TINYTEXT/TEXT/MEDIUMTEXT/LONGTEXT; same for BLOB
ENUM	Up to 65,535 members	N/A	1 or 2 bytes	String	Stores index of value
SET	Up to 64 members	N/A	1–8 bytes	String	Multiple choice list

SQL Server vs MySQL Data Type Comparison

Category	SQL Server	MySQL
Exact Integer	TINYINT (0–255) SMALLINT (-32,768 to 32,767) INT / INTEGER (-2.1B to 2.1B) BIGINT (-2^63 to 2^63-1)	TINYINT (-128 to 127 / 0–255) SMALLINT (-32,768 to 32,767 / 0–65535) MEDIUMINT (-8.3M to 8.3M / 0–16.7M) INT / INTEGER (-2.1B to 2.1B) BIGINT (-2^63 to 2^63-1)
Exact Decimal	DECIMAL(p,s) / NUMERIC(p,s) MONEY SMALLMONEY	DECIMAL(M,D) / NUMERIC(M,D)
Approximate Numeric	FLOAT(n) (n=1–53) REAL (precision 7)	FLOAT(p) (single precision) DOUBLE / REAL (double precision)
Date / Time	DATE DATETIME SMALLDATETIME TIME DATETIME2 DATETIMEOFFSET	DATE DATETIME(fsp) TIMESTAMP(fsp) TIME(fsp) YEAR
Character Strings	CHAR(n) VARCHAR(n) TEXT / NTEXT	CHAR(n) VARCHAR(n) TEXT (TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT)
Unicode Strings	NCHAR(n) NVARCHAR(n) NTEXT	CHAR/VARCHAR/TEXT with utf8/utf8mb4 collation
Binary Data	BINARY(n) VARBINARY(n) IMAGE	BINARY(n) VARBINARY(n) BLOB (TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB)
Other	BIT UNIQUEIDENTIFIER SQL_VARIANT XML	BIT ENUM SET JSON

Example: College Admission Database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

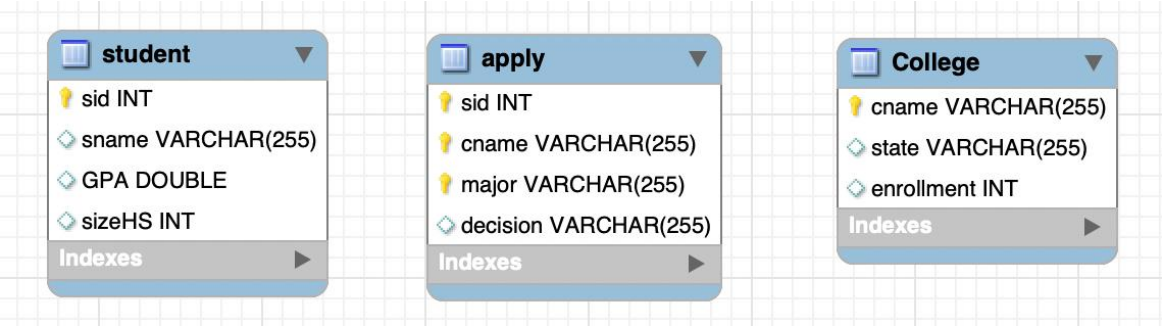
	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

SQL: Create Tables

```
create table college (
        cname varchar(255),
        state varchar(255),
        enrollment int,
        primary key(cname));

create table student(
        sid int,
        sname varchar(255),
        GPA real,
        sizeHS int,
        primary key(sid));
```

```
create table apply (
        sid int,
        cname varchar(255),
        major varchar(255),
        decision varchar(255),
        primary key(sid,cname,major));
```



SQL: Updates Tables

- **Insert into**
  - **insert into** *instructor* **values** (‘10211’, ’Smith’, ’Biology’, 66000);
- **Delete from**
  - Remove all tuples from the *student* relation
    - **delete from** *student*
- **Update table**
  - **Update Table**
  - **Set A_1=$Expr_1$,A_2=$Expr_2$,…,A_n=$Expr_n$**
  - **Where Condition**
- **Drop Table**
  - **drop table** *r*

SQL: Show, Describe, Drop Tables

- Table Creation and Deletion

- drop table student;
- drop table if exists student;
- SHOW TABLES;
- DESCRIBE student;
- SHOW COLUMNS FROM student;

SQL: Insert Records

```
create table college(
        cname varchar(255),
        state varchar(255),
        enrollment int,
        primary key(cname));
```

- Insert a new college

```
insert into College values ('Carnegie Mellon', 'PA', 11500);
```

SQL: Insert Records

- Insert a new college

```
insert into College values ('Carnegie Mellon', 'PA', 11500);
```

- Insert a set of colleges

```
INSERT college (cname,state,enrollment)
VALUES
        ('Stanford','CA',15000),
        ('Berkeley','CA',36000),
        ('MIT','MA',10000),
        ('Cornell','NY',21000);
```

SQL: Select statement

- The **select** lists the attributes in the result of a query
  - Corresponds to the **projection operation** of the relational algebra
  - **Example**: find the names of all instructors:
  - Algebra

$$\pi_{Name}(instructor)$$

  - SQL

```
select name
from instructor
```

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

- SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
  - E.g., *Name* ≡ *NAME* ≡ *name*

SQL: Select statement

- **Select** with condition
  - Corresponds to the combination of **projection and selection operations** of the relational algebra
  - **Example**: find the names of instructors from Physics dept:
  - Algebra

$$\pi_{Name}(\sigma_{dept_name = \text{'Physics'}}(instructor))$$

  - SQL

```
select name
from instructor
Where dept_name = 'Physics'
```

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

SQL: Select statement

```
/*************************************************************
  IDs, names, and GPAs of students with GPA > 3.6
*************************************************************/

select sID, sName, GPA
from Student
where GPA > 3.6;

/*** Same query without GPA ***/

select sID, sName
from Student
where GPA > 3.6;
```

	sID	sName	GPA
1	123	Amy	3.9
2	456	Doris	3.9
3	654	Amy	3.9
4	678	Fay	3.8
5	876	Irene	3.9
6	987	Helen	3.7

SQL: Expression, Rename result attribute

```
/*****************************************************
  Add scaled GPA based on sizeHS
  Also note missing Where clause
*****************************************************

select s_id, s_name, GPA, sizeHS,
GPA*(sizeHS/1000.0)
from Students;

/*** Rename result attribute ***/

select s_id, s_name, GPA, sizeHS,
GPA*(sizeHS/1000.0) as scaledGPA
from Students;
```

	sID	sName	GPA	sizeHS	scaledGPA
1	123	Amy	3.9	1000	3.9
2	234	Bob	3.6	1500	5.3999996
3	345	Craig	3.5	500	1.75
4	456	Doris	3.9	1000	3.9
5	543	Craig	3.4	2000	6.8
6	567	Edward	2.9	2000	5.8
7	654	Amy	3.9	1000	3.9
8	678	Fay	3.8	200	0.76
9	765	Jay	2.9	1500	4.3500004
10	789	Gary	3.4	800	2.72
11	876	Irene	3.9	400	1.5600001
12	987	Helen	3.7	800	2.96

SQL: %, * operators

```
/*************************************************************
  All applications to bio
*************************************************************/
select sID, major
from Apply
where major like '%bio%';

/*** Same query with Select * ***/
select *
from Apply
where major like '%bio%';
```

	sID	major
1	234	biology
2	345	bioengineering
3	345	bioengineering
4	876	biology
5	876	marine biology

	sid	cname	major	decision
1	234	Berkeley	biology	N
2	345	Cornell	bioengineering	N
3	345	MIT	bioengineering	Y
4	876	MIT	biology	Y
5	876	MIT	marine biology	N

SQL: Select statement

- A typical SQL query has the form:

select A_1, A_2, ..., A_n
from r_1, r_2, ..., r_m
where P

  - A_i represents an attribute
  - R_i represents a relation
  - P is a predicate.
- The result of an SQL query is a relation.

SQL: Select statement

The Basic SELECT Statement

3 Select $A_1, A_2, \ldots, A_n$ ← what to return

1 From $R_1, R_2, \ldots, R_m$ ← relations

2 Where condition ← combine filter

$$\Pi_{A_1,\ldots,A_n}\left(\sigma_{condition}\left(R_1 \times \ldots \times R_m\right)\right)$$

SQL: Select statement

```
Select A1,A2,…,An
From   R1,R2, …,Rm
Where  condition
```

Demo: simple college admissions database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

SQL: Delete and Drop Records

- Delete one row

```
DELETE FROM students
WHERE name = 'David';
```

- Delete with condition

```
DELETE FROM students
WHERE gpa < 3.0;
```

- Delete all rows (table become empty)

```
DELETE FROM students;
```

- DROP (remove table (structure + data))

```
DROP TABLE students;
```